TUGAS PRAKTIKUM 4

PEMPROGRAMAN MOBILE



Disusun Oleh :

Nama : Salu Rahmawati

NIM :20240801001

Fakultas : Ilmu komputer

PROGRAM STUDI TEKNIK INFORMATIKA

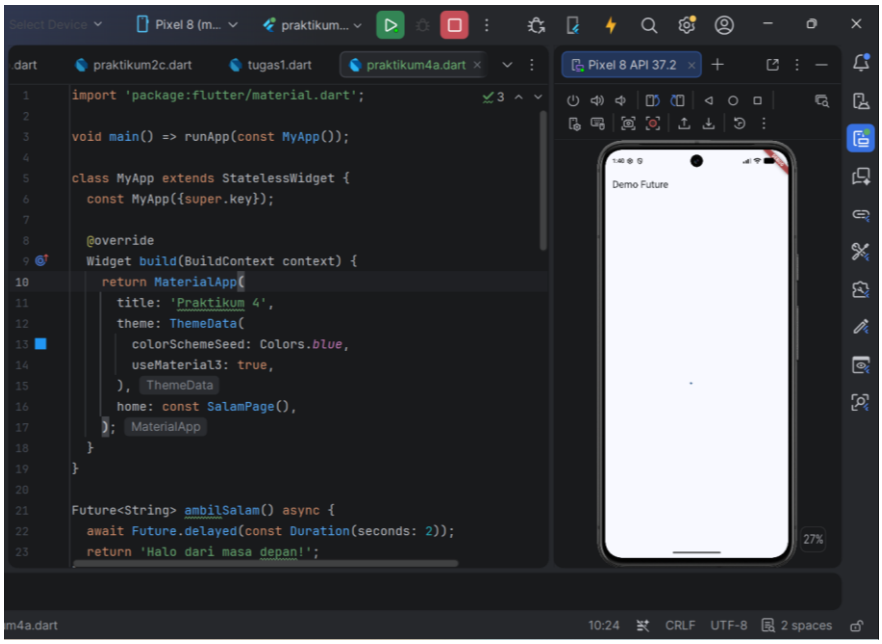
FAKULTAS ILMU KOMPUTER

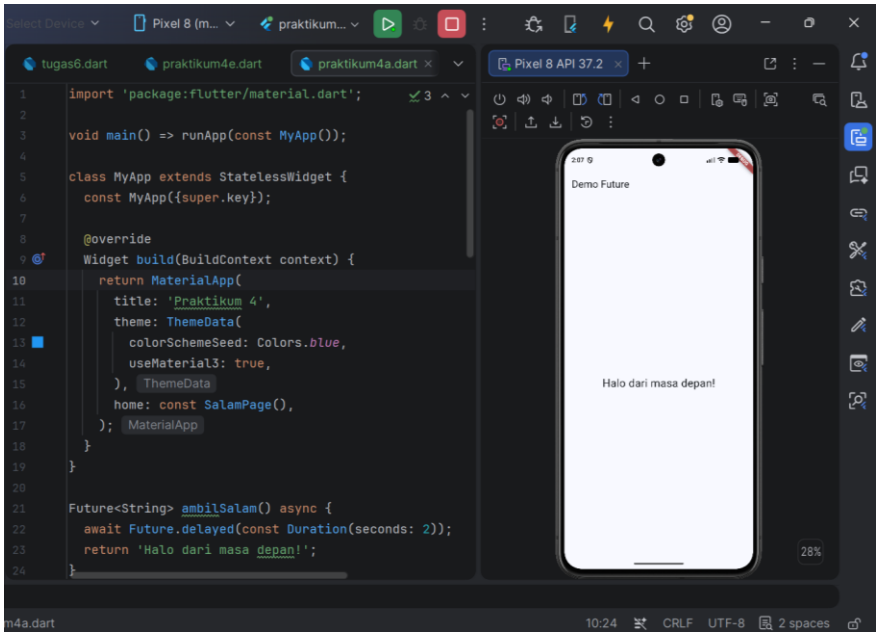
UNIVERSITAS ESA UNGGUL TANGERANG

2026

A. Mengenal Future dan FutureBuilder

Pada tahap pertama dibuat fungsi ambilSalam() yang mensimulasikan proses pengambilan data selama dua detik. Hasil dari fungsi tersebut kemudian ditampilkan menggunakan FutureBuilder. Selama proses berlangsung, aplikasi menampilkan indikator loading dan setelah selesai akan menampilkan hasil data.





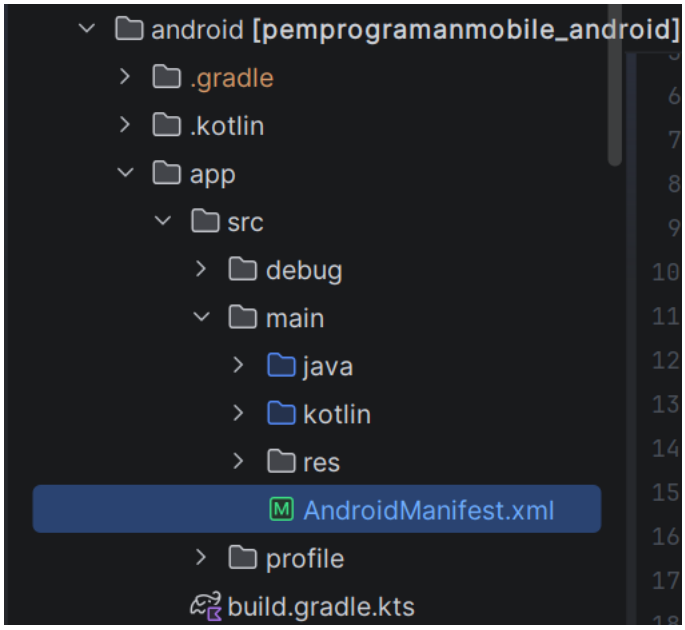
B. Menyiapkan Package HTTP

ackage http digunakan untuk melakukan request ke REST API. Package tersebut ditambahkan melalui terminal menggunakan perintah:

```
C:\pemprogramanmobile>flutter pub add http
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
+ http 1.6.0
+ http_parser 4.1.2
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.14 available)
+ typed_data 1.4.0
  vector_math 2.4.0 (2.4.3 available)
+ web 1.1.1
Changed 4 dependencies!
5 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.

C:\pemprogramanmobile>
```

Selain itu, pada Android ditambahkan permission:




```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>
    <application
```

Permission tersebut diperlukan agar aplikasi dapat mengakses internet.

C. Membuat Model dan Mengambil Data

Data pengguna dari endpoint /users dibuat menjadi model Pengguna. Model memiliki beberapa atribut seperti id, name, email, phone, dan website. Data JSON kemudian diubah menjadi objek Pengguna menggunakan constructor fromJson.

Pengambilan data dilakukan menggunakan:

```dart
Future<List<Pengguna>> ambilPengguna() async {
  final uri = Uri.parse(
    'https://jsonplaceholder.typicode.com/users',
  );

  final response = await http.get(uri).timeout(
    const Duration(seconds: 10),
  );

  if (response.statusCode != 200) {
    throw Exception(
      'Gagal memuat data (kode ${response.statusCode})',
    );
  }

  final List<dynamic> data = jsonDecode(response.body);

  return data
      .map((e) => Pengguna.fromJson(e as Map<String, dynamic>))
      .toList();
}
```

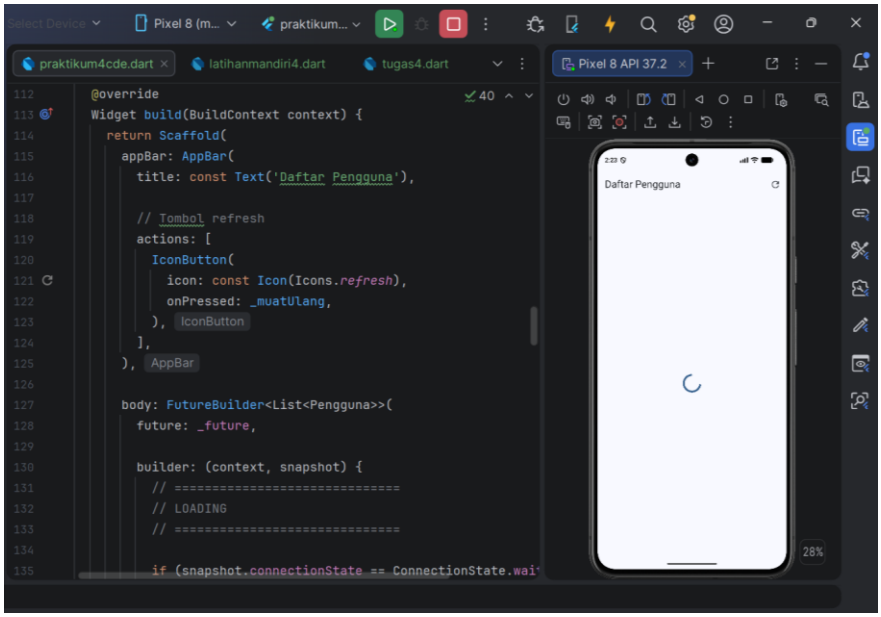
Kemudian response diperiksa. Jika status code bukan 200, program akan menghasilkan error. Jika berhasil, data JSON diubah menjadi objek Pengguna.

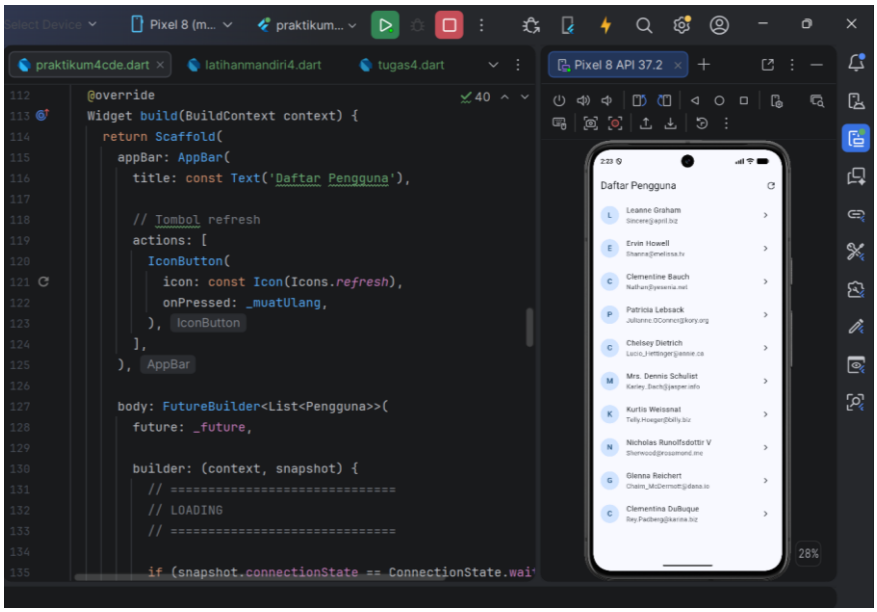
D. Menampilkan Data dengan FutureBuilder

Data pengguna ditampilkan menggunakan FutureBuilder<List<Pengguna>>. Program memiliki tiga kondisi utama, yaitu:

- Loading → menampilkan CircularProgressIndicator.
- Error → menampilkan pesan kesalahan dan tombol Coba lagi.
- Data → menampilkan daftar pengguna menggunakan ListView.builder.

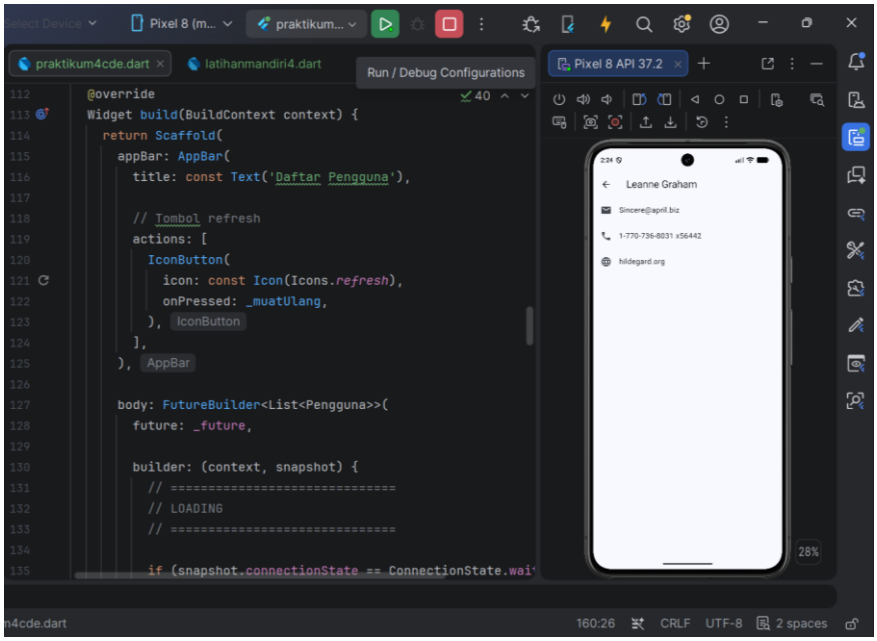
Tombol refresh juga disediakan untuk memuat ulang data dari API.





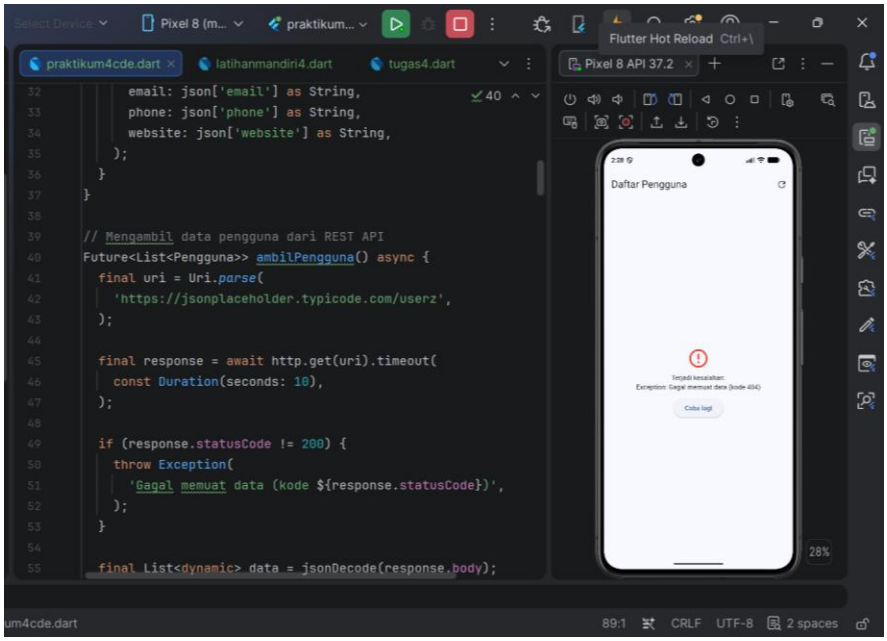
E. Halaman Detail

Ketika salah satu pengguna dipilih, aplikasi membuka halaman detail. Halaman tersebut menampilkan informasi berupa email, nomor telepon, dan website pengguna.

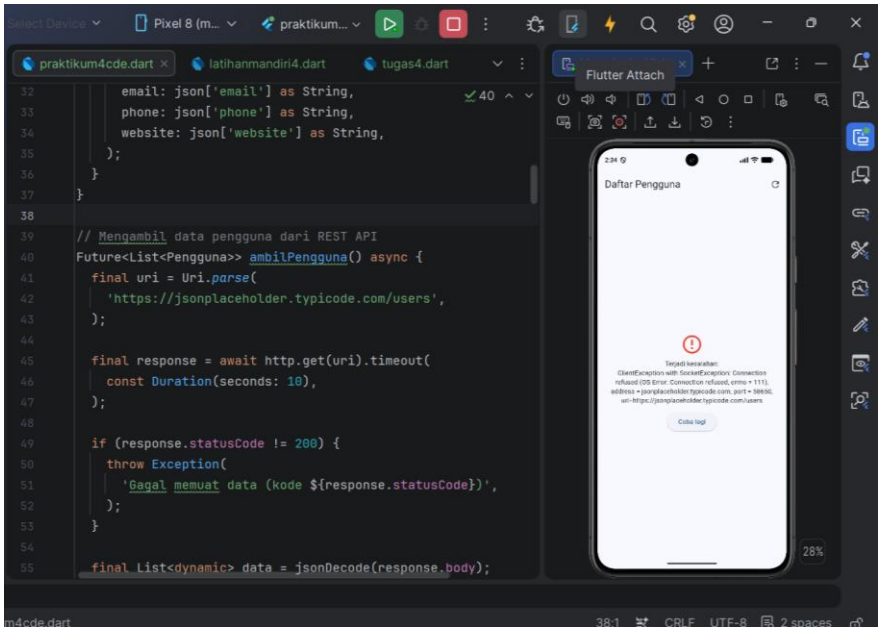


F. Percobaan Penanganan Error

Pada praktikum dilakukan percobaan dengan mengubah endpoint menjadi URL yang salah sehingga menghasilkan status 404. Setelah URL dikembalikan ke URL yang benar, data dapat dimuat kembali menggunakan tombol Coba lagi.

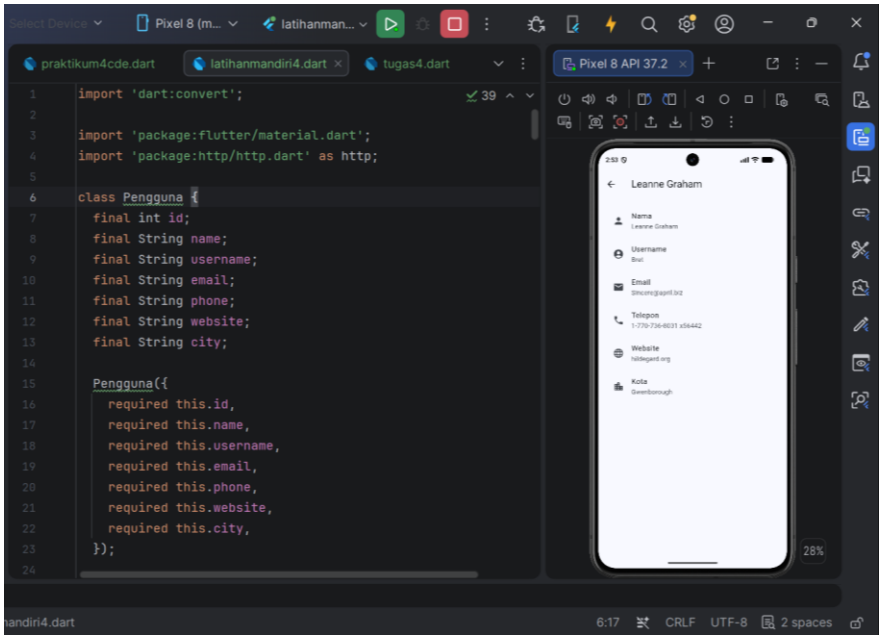


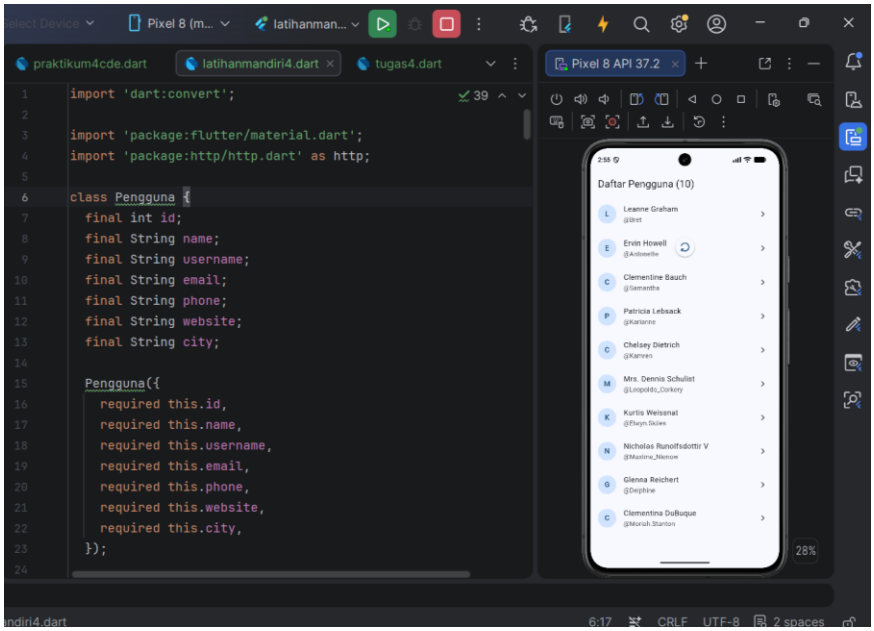
Percobaan kedua dilakukan dengan mematikan koneksi internet. Ketika aplikasi melakukan request, akan muncul error jaringan. Setelah koneksi internet diaktifkan kembali, data dapat dimuat ulang menggunakan tombol **Coba lagi**.



Latihan Mandiri

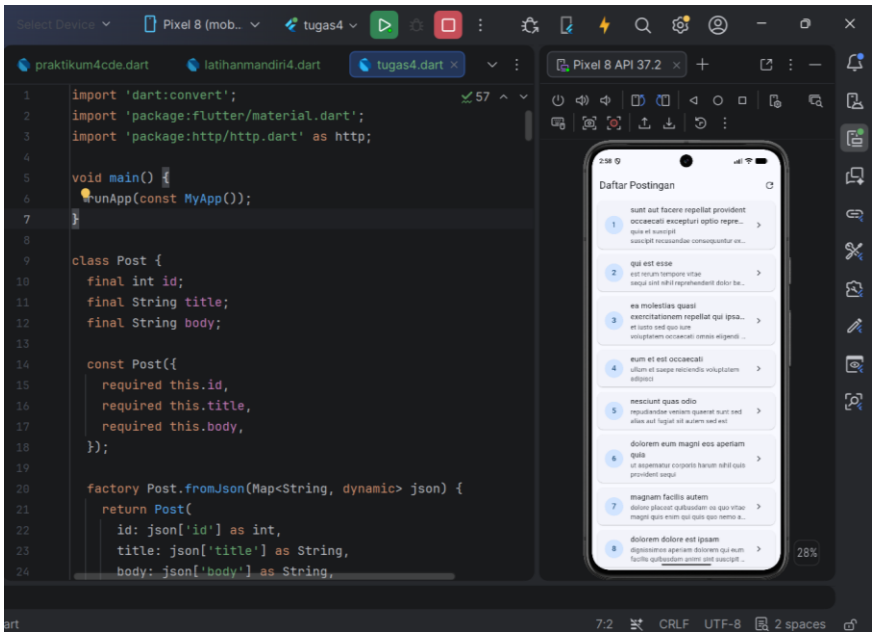
Pada Latihan Mandiri Pertemuan 4, aplikasi Flutter dikembangkan untuk mengambil data pengguna dari REST API dan menampilkannya dalam bentuk daftar. Data pengguna dilengkapi dengan username dan kota, serta aplikasi dapat melakukan refresh data dengan menarik layar ke bawah. Aplikasi juga menangani kondisi ketika data kosong dengan menampilkan “Tidak ada data” dan menampilkan jumlah pengguna pada AppBar setelah data berhasil dimuat.

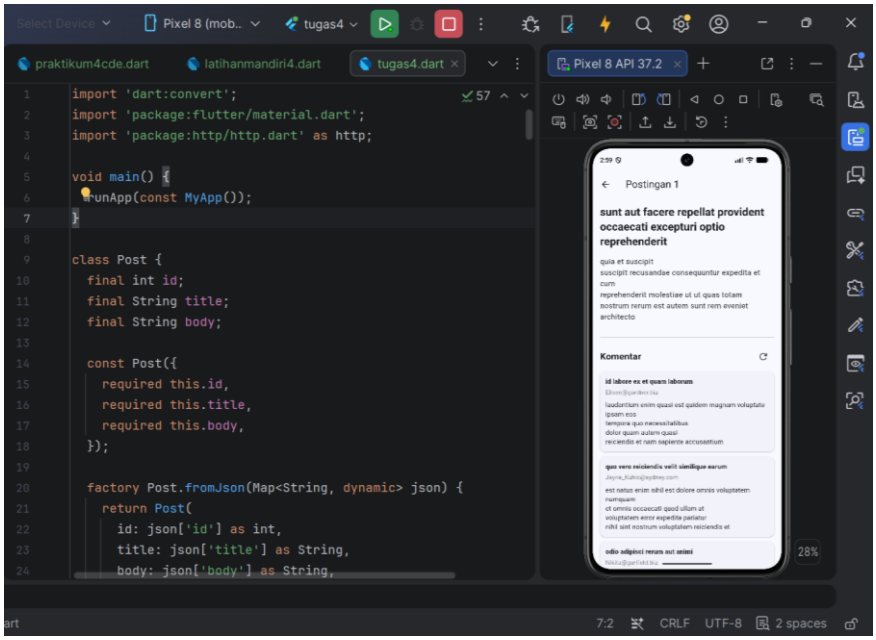




Tugas Pertemuan 4

Pada tugas Pertemuan 4, dibuat aplikasi Daftar Postingan yang mengambil data postingan dari REST API menggunakan FutureBuilder. Data postingan ditampilkan dalam bentuk daftar, kemudian pengguna dapat memilih salah satu postingan untuk melihat halaman detail beserta komentar yang diambil dari endpoint API berdasarkan ID postingan. Aplikasi juga menerapkan model Post dan Komentar, proses loading dan error handling, serta tombol untuk mencoba kembali ketika terjadi kesalahan.





Refleksi

1. Apa yang terjadi bila Future dibuat di dalam build() dan bukan di initState()? Mengapa?

Jika Future dibuat di dalam build(), proses mengambil data dapat dijalankan berulang kali setiap kali widget melakukan rebuild. Hal ini dapat menyebabkan request API berulang dan tidak efisien. Karena itu, Future sebaiknya dibuat di initState() agar tidak dibuat ulang setiap rebuild.

2. Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode? Mengapa keduanya diperlukan?

response.statusCode digunakan untuk mengetahui apakah request HTTP berhasil berdasarkan kode status dari server, sedangkan snapshot.hasError digunakan untuk mengetahui apakah terjadi error pada proses Future, misalnya masalah koneksi atau exception. Keduanya diperlukan karena menangani jenis kesalahan yang berbeda.

3. Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai Map?

Karena class model membuat struktur data lebih jelas dan terorganisir. Data dapat diakses menggunakan properti seperti pengguna.name sehingga kode lebih mudah dibaca, dikelola, dan dikembangkan.

4. Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?

Karena pengambilan data dari internet membutuhkan waktu dan dapat mengalami kegagalan. Status loading memberi informasi bahwa data sedang diproses, sedangkan status error memberi informasi ketika terjadi masalah sehingga pengguna tidak hanya melihat halaman kosong.

Kesimpulan

Praktikum 4 memberikan pemahaman tentang cara mengambil dan menampilkan data dari REST API pada Flutter menggunakan http, Future, async/await, FutureBuilder, dan JSON. Data dari API diubah menjadi class model agar lebih mudah dikelola, kemudian ditampilkan pada halaman aplikasi dengan penanganan kondisi loading, error, dan data. Melalui latihan dan tugas, juga dipelajari penggunaan API untuk menampilkan data pengguna, postingan, serta komentar secara dinamis.